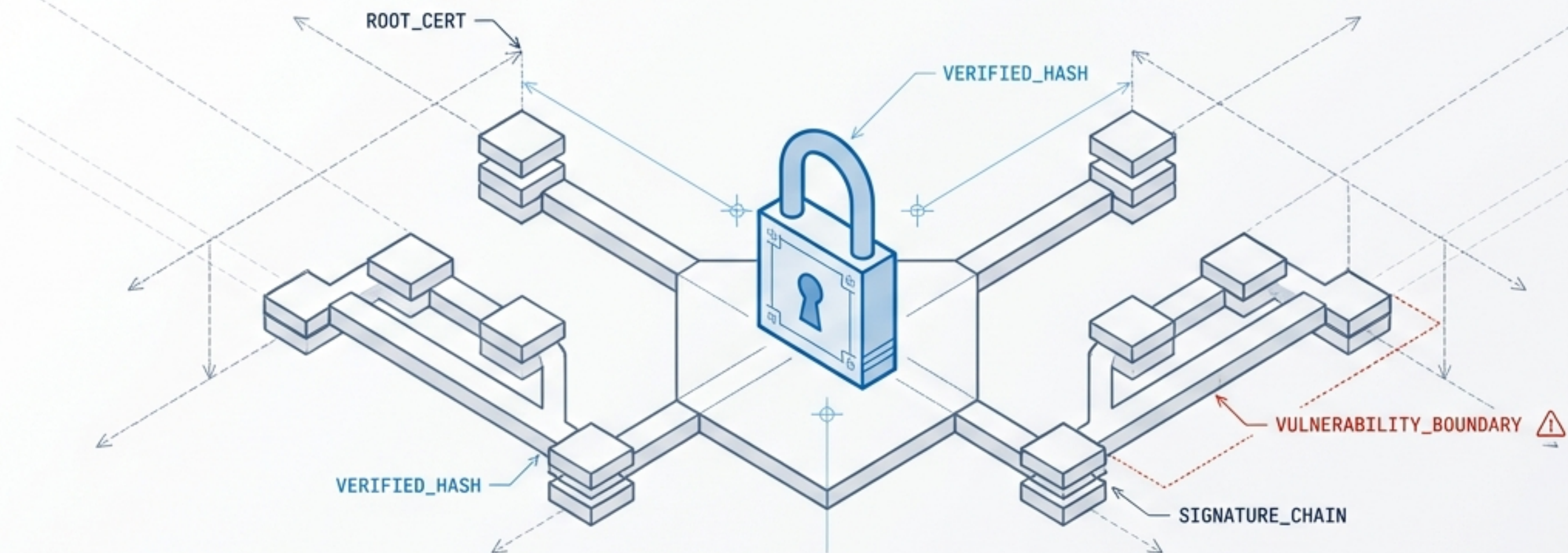




Provenance Through PKI

Cryptographic Truth in Version Control

v0.13.30 | 14 March 2026 | SGraph Send

Git Authorship is Metadata, Not Proof



The Illusion of Identity

In legacy Git, authorship is trivially forgeable. Anyone can set their configuration to impersonate any user.



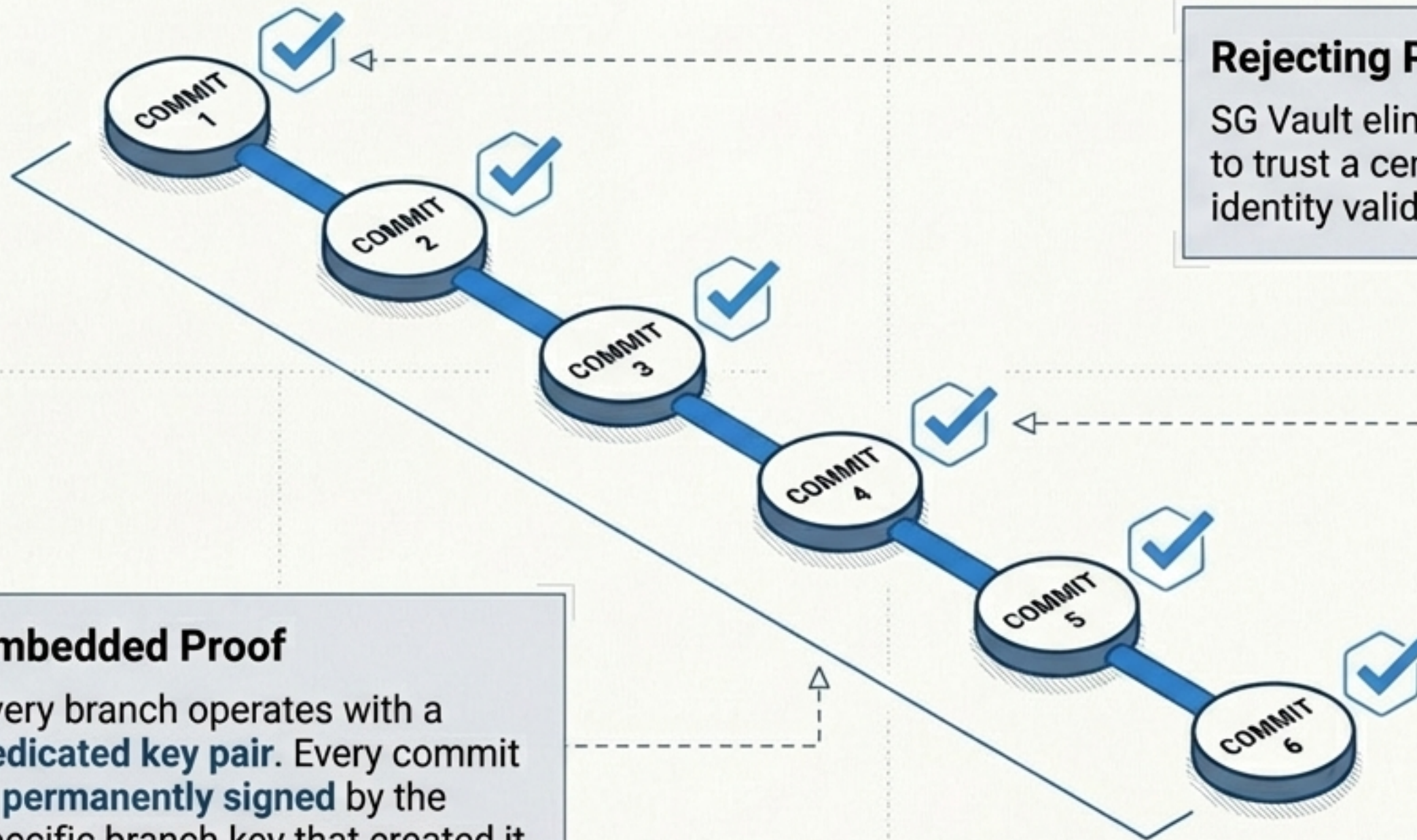
The Platform Vulnerability

Platforms map commits to accounts via email matching. This relies entirely on platform trust, not **cryptographic proof**.

The Core Risk

If the central platform is ever compromised, all historical authorship claims become completely **unreliable**.

Cryptographically Attributable History



Rejecting Platform Trust

SG Vault eliminates the need to trust a central server for identity validation.

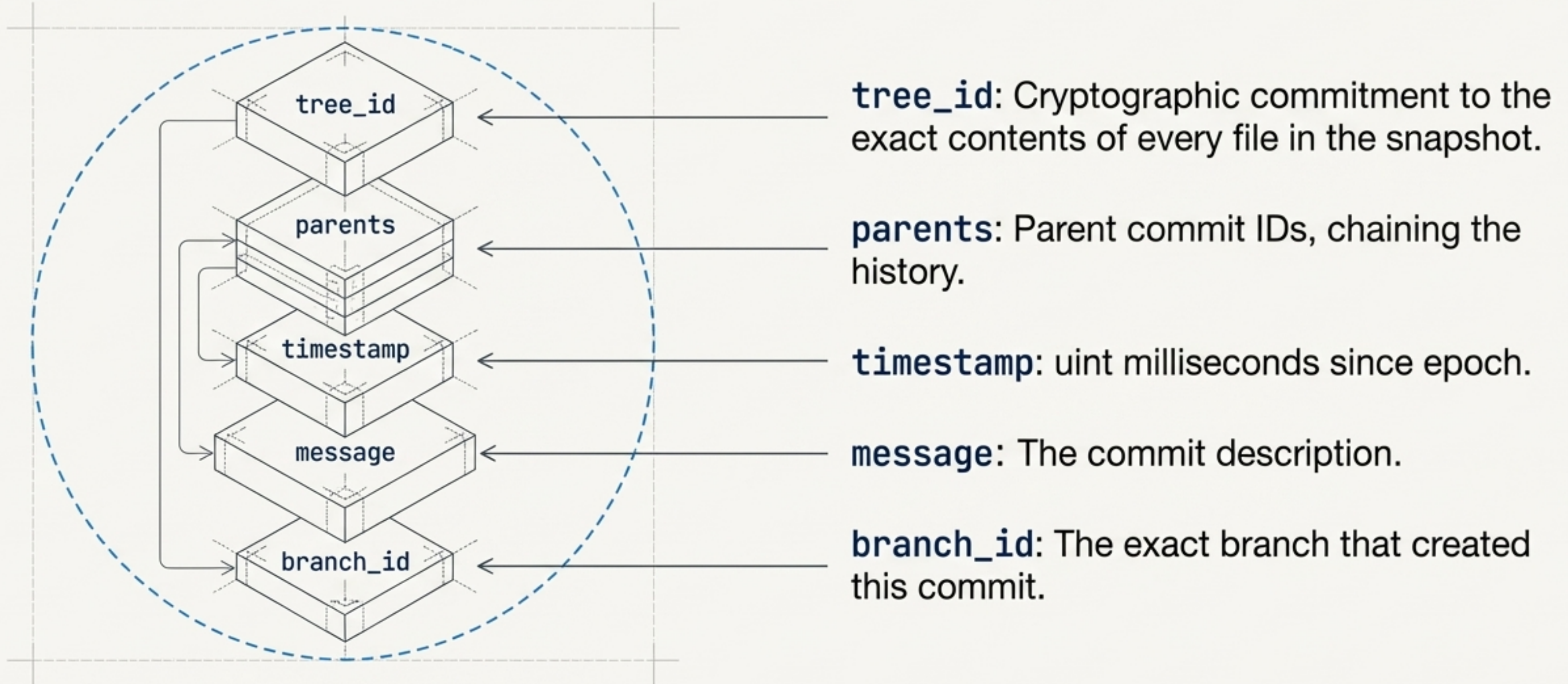
Embedded Proof

Every branch operates with a **dedicated key pair**. Every commit is **permanently signed** by the specific branch key that created it.

The History is the Proof

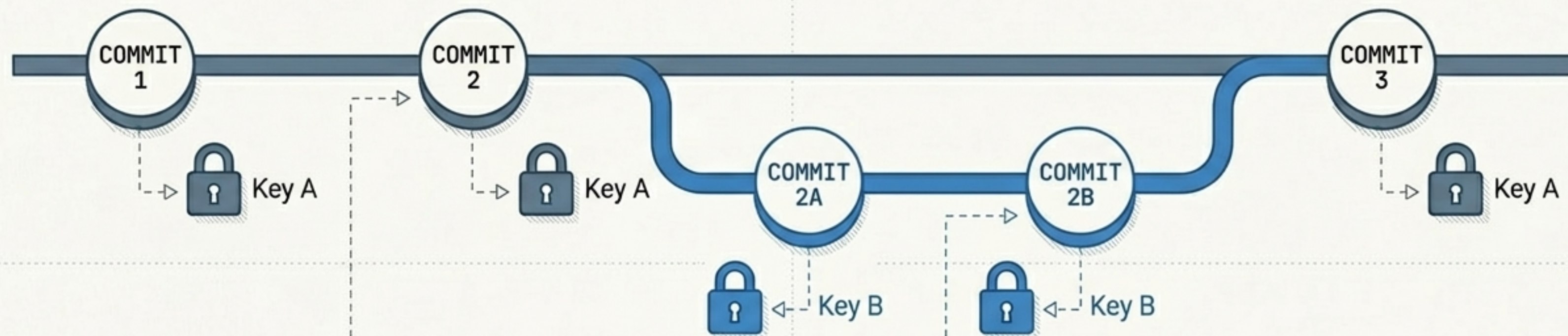
The signature chain is not stored in a separate database; it is embedded directly into the structural graph of the commits.

Anatomy of a Signed Commit



The Tamper-Proof Guarantee: If any single field changes after signing, the signature instantly invalidates. Tampering with any historical commit fundamentally destroys all descendant commit IDs.

Tracing the Cryptographic Chain



Named Branch Origins

A named branch's signature chain originates entirely at its first merge. All subsequent commits on the named branch are signed by the named branch key.

Clone Isolation

All commits on a clone branch are signed exclusively by that clone's unique key.

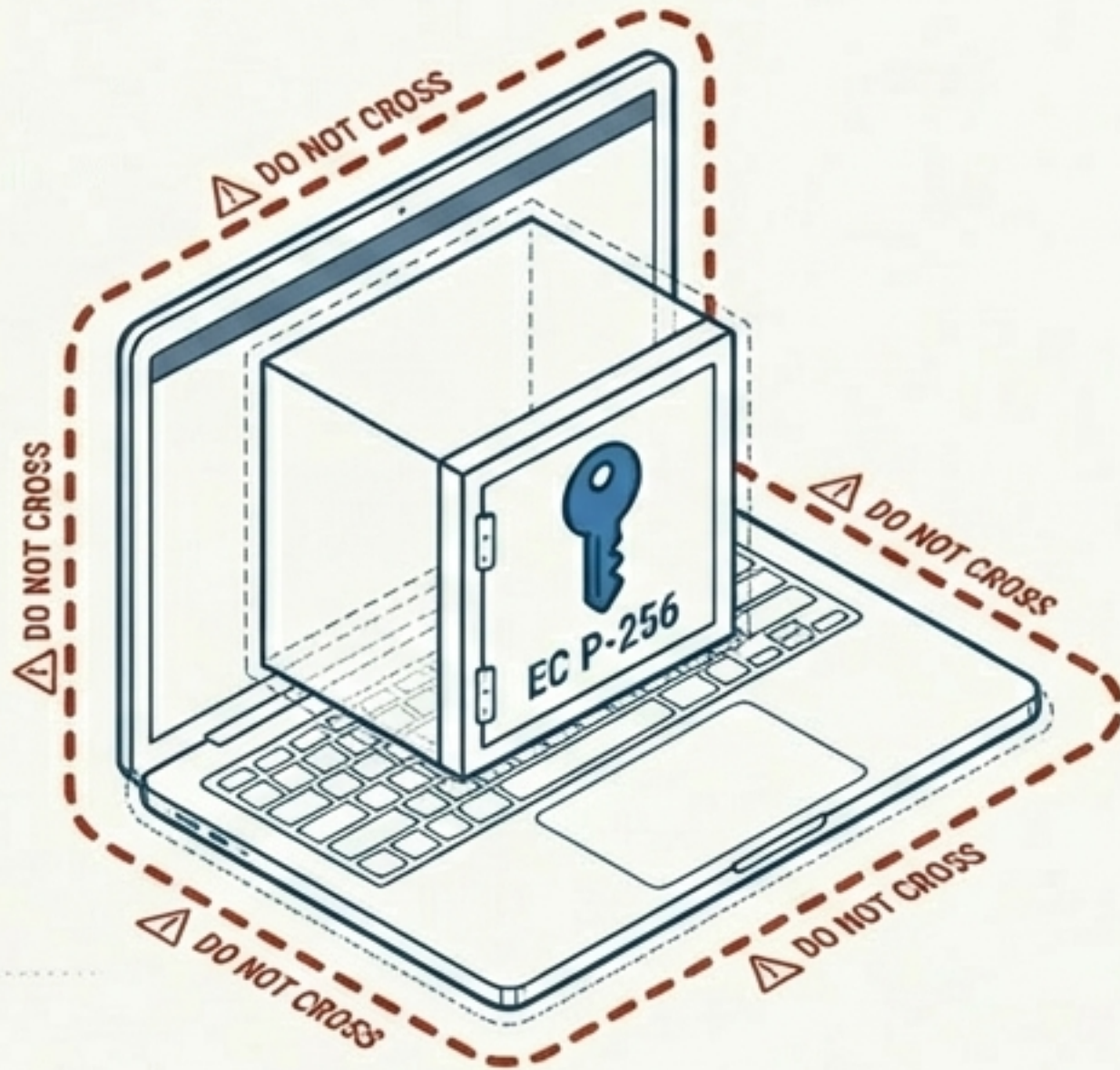
Immediate Verification

Inspecting any commit in the graph instantly reveals three absolute truths:

1. Which **branch** created it.
2. Which **exact key** signed it.
3. Whether the signature remains mathematically valid.

The Clone is the Identity

Automatic Generation: Every `clone()` automatically generates an EC P-256 key pair. The branch is the clone's identity.



Strict Provenance

Every `commit` traces directly to a **branch** → **key pair** → **physical device**.

Total Isolation

You can only sign commits for your own **branch**.

Zero Conflicts

Your **branch** is yours alone, meaning a remote push always succeeds with zero conflicts.

Default Anonymity

The key proves 'same **entity**' across multiple commits without forcefully revealing human identity.

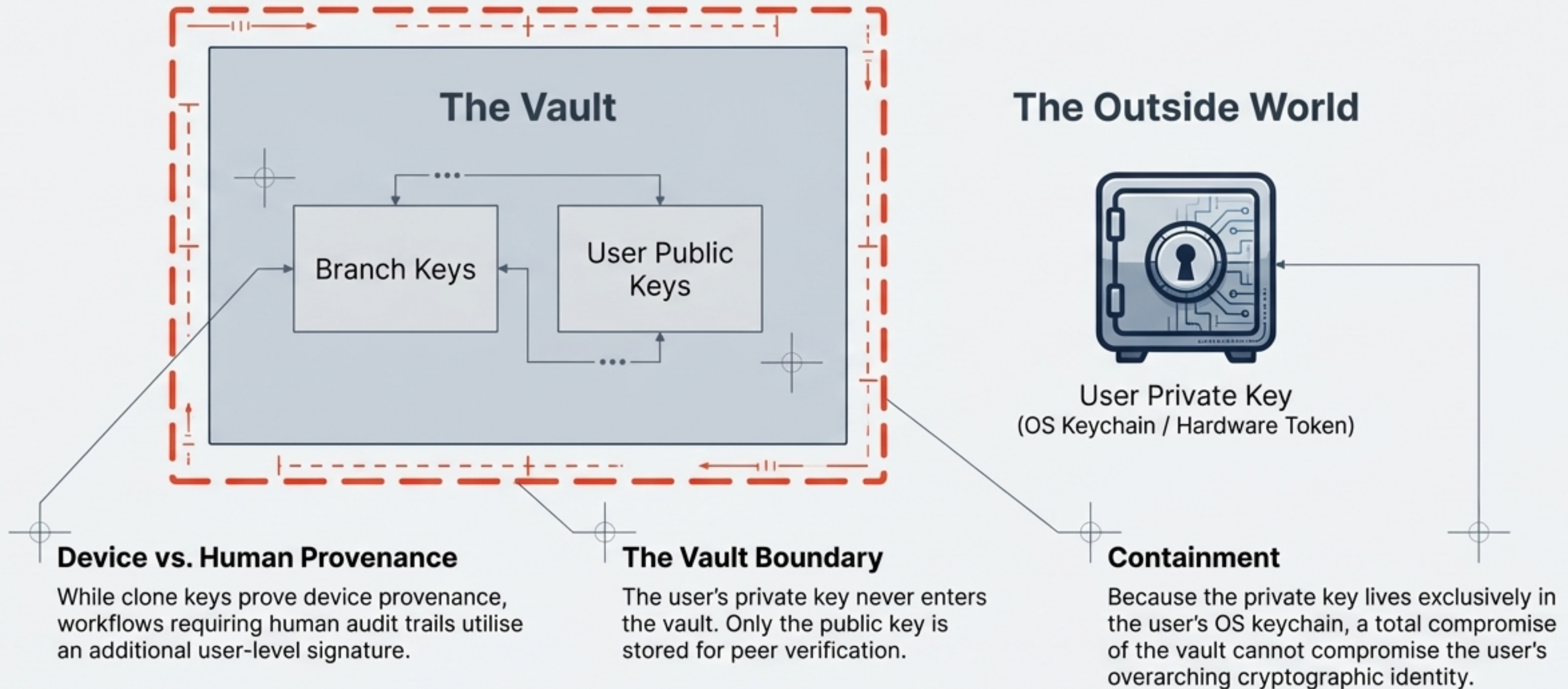
The Three Tiers of Private Keys

Key Type	Leaves Device?	Stored Where	Loss Condition
Clone Branch Key	Never	<code>local/branch_key.pem</code>	Branch permanently locked (normal lifecycle event, history remains intact).
Named Branch Key	In vault (encrypted)	<code>bare/keys/key-{id}</code>	Recoverable with the vault key.
User Key	Never in the vault	<code>~/.sg-send/keys/</code> (user-managed)	User re-imports from their own backup.

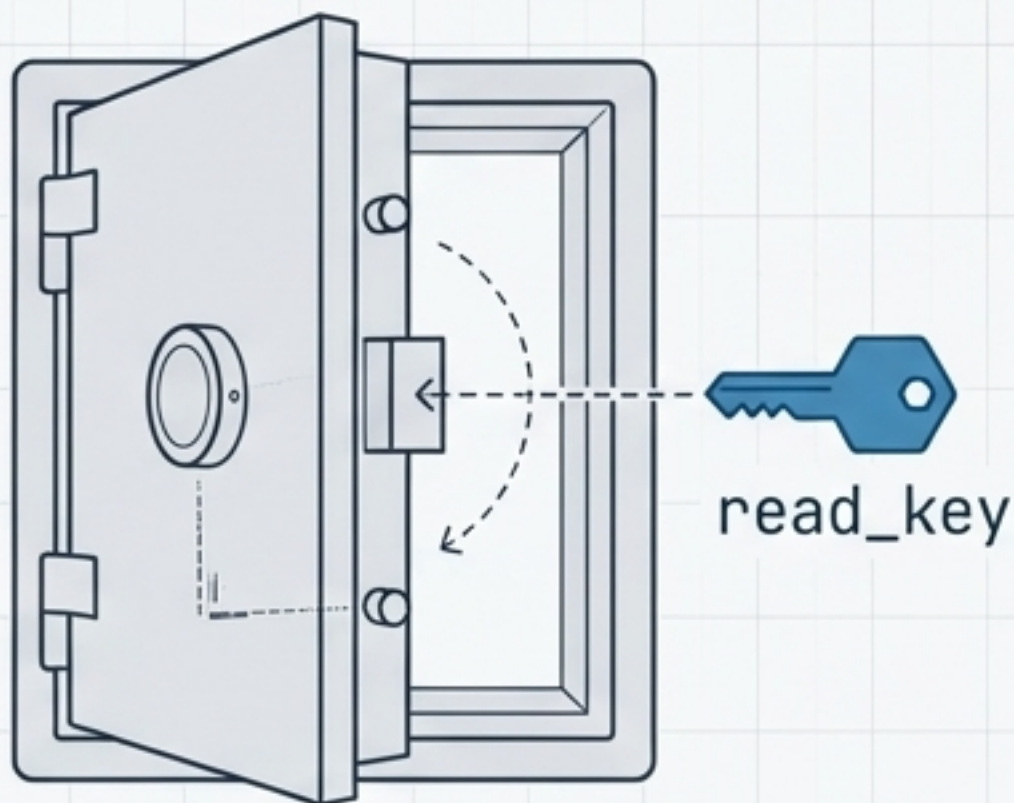


Diagnostic Note: The **clone key** is **deliberately local-only**. If the same key appeared on two devices, precise device-level provenance would be irreparably broken.

Separating Vault Compromise from Identity Compromise

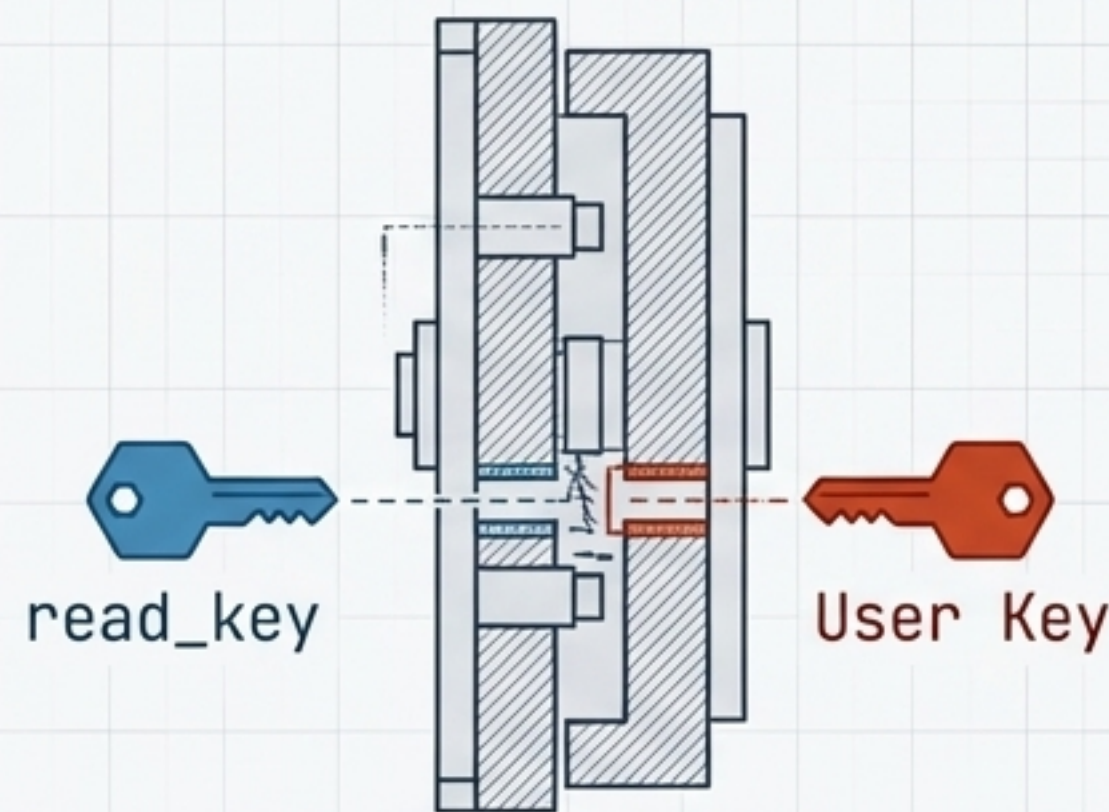


Governing the Named Branch



Open (Team Collaboration)

Encrypted purely with the vault's read_key. Anyone holding the vault key can decrypt the named branch private key and perform merges.



Locked (Strict Custody)

Additionally wrapped with a specific user's public key. Only that specific user can decrypt the named branch key to execute a merge.

Unforgeable Governance: Even an administrator holding the vault key cannot forge a merge into a locked named branch without the specific user's key.

Zero-Trust Server Architecture

VERIFIED SIGNATURE CHAIN 



LOCAL USER



PHYSICAL
DISCONNECT



SERVER
(OPAQUE BLOB STORAGE)

Local Verification

No server calls required. Users decrypt commits, read signatures, and verify against public keys entirely locally.

Server Blindness

Servers only store opaque encrypted blobs. They cannot identify commit objects, let alone forge valid mathematical signatures.

Total Resilience

History integrity survives absolute server compromise. Attackers can delete data (denial of service) but cannot forge valid signed commits.

Air-gapped Audit

An auditor can verify the entire cryptographic history of a vault completely offline from a simple zip export.

What True Provenance Enables

